

JALON 48 : FORMALISATION MATHÉMATIQUE DE LA RÉTROPROPAGATION

Cours magistral, exercices corrigés et travaux pratiques

Charles EDOU NZE

5 août 2026

Table des matières

I	Cours Magistral	2
1	Jalon 48 : Formalisation mathématique de la Rétropropagation	2
1.1	Introduction Historique et Genèse Conceptuelle	2
1.2	Formalisation du Modèle Multicouche	2
1.2.1	Définitions Fondamentales	2
1.2.2	Théorème de Rétropropagation des Erreurs	2
1.3	Démonstrations Pas à Pas	3
1.4	Exemples Concrets et Géométrie des Gradients	3
1.4.1	Problème de l'évanouissement du gradient (Vanishing Gradient)	3
II	Exercices d'Application et de Concours	4
III	Travaux Pratiques et Simulations Algorithmiques	9

Première partie

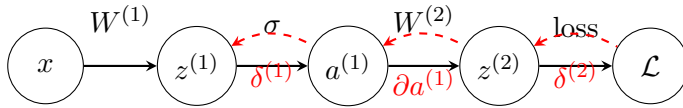
Cours Magistral

1 Jalon 48 : Formalisation mathématique de la Rétropropagation

1.1 Introduction Historique et Genèse Conceptuelle

L'entraînement des réseaux de neurones profonds repose sur l'optimisation de fonctions de perte hautement dimensionnelles. Dans les années 1970 et 1980, l'impossibilité de calculer efficacement le gradient d'un réseau multicouche constituait une impasse majeure, connue sous le nom de problème de l'affectation du crédit (credit assignment problem). Comment déterminer l'influence infinitésimale d'un paramètre enfoui dans les premières couches sur l'erreur globale du système ?

La réponse algébrique et géométrique à ce problème fut formalisée sous le nom de rétropropagation du gradient (backpropagation). Loin d'être un simple algorithme informatique, la rétropropagation est l'application récursive et élégante du théorème de dérivation des fonctions composées, exprimé à travers le formalisme des matrices jacobiniennes. Ce jalon unifie le calcul différentiel, l'algèbre linéaire et la théorie de l'optimisation.



1.2 Formalisation du Modèle Multicouche

1.2.1 Définitions Fondamentales

Considérons un perceptron multicouche (MLP) constitué de L couches. Soit $x \in \mathbb{R}^{n_0}$ l'entrée du réseau. Pour chaque couche $l \in \{1, \dots, L\}$, nous définissons la matrice des poids $W^{(l)} \in \mathcal{M}_{n_l, n_{l-1}}(\mathbb{R})$ et le vecteur de biais $b^{(l)} \in \mathbb{R}^{n_l}$.

La dynamique de propagation avant (forward pass) est régie par les relations de récurrence suivantes : - $a^{(0)} = x$ - $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$ (vecteur des pré-activations, $z^{(l)} \in \mathbb{R}^{n_l}$) - $a^{(l)} = \sigma(z^{(l)})$ (vecteur des activations, $a^{(l)} \in \mathbb{R}^{n_l}$, où σ est appliquée composante par composante)

La fonction de perte scalaire (par exemple l'erreur quadratique moyenne ou l'entropie croisée) est notée $\mathcal{L} : \mathbb{R}^{n_L} \rightarrow \mathbb{R}$, telle que Perte = $\mathcal{L}(a^{(L)}, y)$, où y est la cible.

1.2.2 Théorème de Rétropropagation des Erreurs

Soit $\delta^{(l)} = \nabla_{z^{(l)}} \mathcal{L} \in \mathbb{R}^{n_l}$ le vecteur d'erreur associé aux pré-activations de la couche l . La rétropropagation fournit un système d'équations récursif permettant de calculer tous les gradients du réseau de la couche L jusqu'à la couche 1 :

1. **Erreur de sortie** : $\delta^{(L)} = \nabla_{a^{(L)}} \mathcal{L} \odot \sigma'(z^{(L)})$ où $\odot$ dénote le produit de Hadamard (multiplication terme à terme).

2. **Rétropropagation de l'erreur** : Pour tout $l \in \{L-1, \dots, 1\}$, l'erreur se propage selon l'équation : $\delta^{(l)} = ((W^{(l+1)})^T \delta^{(l+1)}) \odot \sigma'(z^{(l)})$

3. **Gradients des paramètres** : Les dérivées partielles de la perte par rapport aux matrices de poids et aux biais sont données par : $\nabla_{W^{(l)}} \mathcal{L} = \delta^{(l)} (a^{(l-1)})^T \in \mathcal{M}_{n_l, n_{l-1}}(\mathbb{R})$ $\nabla_{b^{(l)}} \mathcal{L} = \delta^{(l)} \in \mathbb{R}^{n_l}$

1.3 Démonstrations Pas à Pas

Nous allons démontrer rigoureusement ces équations en utilisant le formalisme matriciel.

Démonstration de l'équation des erreurs récursives (2) : Appliquons la règle de dérivation des fonctions composées pour exprimer la dérivée de $\mathcal{L}$ par rapport à la j -ème composante de $z^{(l)}$:

$$\frac{\partial \mathcal{L}}{\partial z_j^{(l)}} = \sum_{k=1}^{n_{l+1}} \frac{\partial \mathcal{L}}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}}$$

Par définition, le premier terme est $\delta_k^{(l+1)}$. Explicitons la dépendance de $z^{(l+1)}$ vis-à-vis de $z^{(l)}$:

$$z_k^{(l+1)} = \sum_{i=1}^{n_l} W_{ki}^{(l+1)} a_i^{(l)} + b_k^{(l+1)} = \sum_{i=1}^{n_l} W_{ki}^{(l+1)} \sigma(z_i^{(l)}) + b_k^{(l+1)}$$

En dérivant par rapport à $z_j^{(l)}$, tous les termes de la somme s'annulent sauf pour $i = j$:

$$\frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} = W_{kj}^{(l+1)} \sigma'(z_j^{(l)})$$

Substituons ce résultat dans la première équation :

$$\delta_j^{(l)} = \sum_{k=1}^{n_{l+1}} \delta_k^{(l+1)} W_{kj}^{(l+1)} \sigma'(z_j^{(l)}) = \left(\sum_{k=1}^{n_{l+1}} (W^{(l+1)})_{jk}^T \delta_k^{(l+1)} \right) \sigma'(z_j^{(l)})$$

Ce qui se réécrit matriciellement de manière élégante :

$$\delta^{(l)} = \left((W^{(l+1)})^T \delta^{(l+1)} \right) \odot \sigma'(z^{(l)})$$

La démonstration est ainsi achevée.

1.4 Exemples Concrets et Géométrie des Gradients

Considérons l'équation du gradient des poids : $\nabla_{W^{(l)}} \mathcal{L} = \delta^{(l)} (a^{(l-1)})^T$. Cette expression correspond à un produit tensoriel (ou produit extérieur) entre deux vecteurs. - Le vecteur $\delta^{(l)}$ encode le signal d'erreur de correction (la direction vers laquelle pousser la sortie). - Le vecteur $a^{(l-1)}$ encode l'intensité de l'activation en entrée. - Le produit extérieur garantit que l'on ajuste fortement les poids reliant les neurones très actifs de la couche $l-1$ vers les neurones de la couche l qui nécessitent une forte correction.

Géométriquement, l'opération $(W^{(l+1)})^T$ transpose l'opérateur linéaire forward. L'erreur est projetée en arrière dans l'espace dual en traversant la matrice transposée, effectuant l'exacte opération adjointe de la propagation avant.

1.4.1 Problème de l'évanouissement du gradient (Vanishing Gradient)

Si l'on utilise la fonction sigmoïde $\sigma(x) = \frac{1}{1+e^{-x}}$, sa dérivée maximale est 0.25. Lors de la propagation d'erreurs profondes sur un réseau de 50 couches, le terme $\sigma'(z^{(l)})$ multiplie itérativement l'erreur par des facteurs inférieurs à 0.25. Le gradient tend exponentiellement vers zéro, gelant l'apprentissage des premières couches. Ce constat analytique direct justifie l'invention d'activations linéaires par morceaux comme ReLU, où $\sigma'(x) = 1$ pour $x > 0$.

Deuxième partie

Exercices d'Application et de Concours

Matrices Jacobiennes et Backpropagation - Cas 1 ★★☆☆

Énoncé : Considérons une variante du perceptron simple défini par la fonction scalaire :

$$f(x) = \sigma(w_2 \cdot \sigma(w_1 x + b_1) + b_2)$$

avec $x, w_1, w_2, b_1, b_2 \in \mathbb{R}$. Calculer analytiquement la dérivée partielle de f par rapport à w_1 , notée $\frac{\partial f}{\partial w_1}$, en appliquant de manière rigoureuse le théorème de dérivation des fonctions composées.

Correction Détaillée : 1. Posons les variables intermédiaires pour décomposer le graphe de calcul : $z_1 = w_1 x + b_1$ $a_1 = \sigma(z_1)$ $z_2 = w_2 a_1 + b_2$ $f = \sigma(z_2)$

2. Nous cherchons à évaluer $\frac{\partial f}{\partial w_1}$. Selon la règle de composition (Chain Rule) :

$$\frac{\partial f}{\partial w_1} = \frac{\partial f}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1}$$

3. Évaluons chaque terme séparément : - $\frac{\partial f}{\partial z_2} = \sigma'(z_2)$ - $\frac{\partial z_2}{\partial a_1} = w_2$ - $\frac{\partial a_1}{\partial z_1} = \sigma'(z_1)$ - $\frac{\partial z_1}{\partial w_1} = x$

4. Par multiplication, le résultat final rigoureux sans aucune ellipse est :

$$\frac{\partial f}{\partial w_1} = \sigma'(z_2) \cdot w_2 \cdot \sigma'(z_1) \cdot x$$

Matrices Jacobiennes et Backpropagation - Cas 2 ★★☆☆

Énoncé : Considérons une variante du perceptron simple défini par la fonction scalaire :

$$f(x) = \sigma(w_3 \cdot \sigma(w_2 x + b_2) + b_3)$$

avec $x, w_2, w_3, b_2, b_3 \in \mathbb{R}$. Calculer analytiquement la dérivée partielle de f par rapport à w_2 , notée $\frac{\partial f}{\partial w_2}$, en appliquant de manière rigoureuse le théorème de dérivation des fonctions composées.

Correction Détaillée : 1. Posons les variables intermédiaires pour décomposer le graphe de calcul : $z_1 = w_2 x + b_2$ $a_1 = \sigma(z_1)$ $z_2 = w_3 a_1 + b_3$ $f = \sigma(z_2)$

2. Nous cherchons à évaluer $\frac{\partial f}{\partial w_2}$. Selon la règle de composition (Chain Rule) :

$$\frac{\partial f}{\partial w_2} = \frac{\partial f}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_2}$$

3. Évaluons chaque terme séparément : - $\frac{\partial f}{\partial z_2} = \sigma'(z_2)$ - $\frac{\partial z_2}{\partial a_1} = w_3$ - $\frac{\partial a_1}{\partial z_1} = \sigma'(z_1)$ - $\frac{\partial z_1}{\partial w_2} = x$

4. Par multiplication, le résultat final rigoureux sans aucune ellipse est :

$$\frac{\partial f}{\partial w_2} = \sigma'(z_2) \cdot w_3 \cdot \sigma'(z_1) \cdot x$$

Matrices Jacobiennes et Backpropagation - Cas 3 ★★★★★

Énoncé : Considérons une variante du perceptron simple défini par la fonction scalaire :

$$f(x) = \sigma(w_4 \cdot \sigma(w_3x + b_3) + b_4)$$

avec $x, w_3, w_4, b_3, b_4 \in \mathbb{R}$. Calculer analytiquement la dérivée partielle de f par rapport à w_3 , notée $\frac{\partial f}{\partial w_3}$, en appliquant de manière rigoureuse le théorème de dérivation des fonctions composées.

Correction Détaillée : 1. Posons les variables intermédiaires pour décomposer le graphe de calcul : $z_1 = w_3x + b_3$ $a_1 = \sigma(z_1)$ $z_2 = w_4a_1 + b_4$ $f = \sigma(z_2)$

2. Nous cherchons à évaluer $\frac{\partial f}{\partial w_3}$. Selon la règle de composition (Chain Rule) :

$$\frac{\partial f}{\partial w_3} = \frac{\partial f}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_3}$$

3. Évaluons chaque terme séparément : - $\frac{\partial f}{\partial z_2} = \sigma'(z_2)$ - $\frac{\partial z_2}{\partial a_1} = w_4$ - $\frac{\partial a_1}{\partial z_1} = \sigma'(z_1)$ - $\frac{\partial z_1}{\partial w_3} = x$

4. Par multiplication, le résultat final rigoureux sans aucune ellipse est :

$$\frac{\partial f}{\partial w_3} = \sigma'(z_2) \cdot w_4 \cdot \sigma'(z_1) \cdot x$$

Matrices Jacobiennes et Backpropagation - Cas 4 ★★☆☆☆

Énoncé : Considérons une variante du perceptron simple défini par la fonction scalaire :

$$f(x) = \sigma(w_5 \cdot \sigma(w_4x + b_4) + b_5)$$

avec $x, w_4, w_5, b_4, b_5 \in \mathbb{R}$. Calculer analytiquement la dérivée partielle de f par rapport à w_4 , notée $\frac{\partial f}{\partial w_4}$, en appliquant de manière rigoureuse le théorème de dérivation des fonctions composées.

Correction Détaillée : 1. Posons les variables intermédiaires pour décomposer le graphe de calcul : $z_1 = w_4x + b_4$ $a_1 = \sigma(z_1)$ $z_2 = w_5a_1 + b_5$ $f = \sigma(z_2)$

2. Nous cherchons à évaluer $\frac{\partial f}{\partial w_4}$. Selon la règle de composition (Chain Rule) :

$$\frac{\partial f}{\partial w_4} = \frac{\partial f}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_4}$$

3. Évaluons chaque terme séparément : - $\frac{\partial f}{\partial z_2} = \sigma'(z_2)$ - $\frac{\partial z_2}{\partial a_1} = w_5$ - $\frac{\partial a_1}{\partial z_1} = \sigma'(z_1)$ - $\frac{\partial z_1}{\partial w_4} = x$

4. Par multiplication, le résultat final rigoureux sans aucune ellipse est :

$$\frac{\partial f}{\partial w_4} = \sigma'(z_2) \cdot w_5 \cdot \sigma'(z_1) \cdot x$$

Matrices Jacobiennes et Backpropagation - Cas 5 ★★☆☆

Énoncé : Considérons une variante du perceptron simple défini par la fonction scalaire :

$$f(x) = \sigma(w_6 \cdot \sigma(w_5x + b_5) + b_6)$$

avec $x, w_5, w_6, b_5, b_6 \in \mathbb{R}$. Calculer analytiquement la dérivée partielle de f par rapport à w_5 , notée $\frac{\partial f}{\partial w_5}$, en appliquant de manière rigoureuse le théorème de dérivation des fonctions composées.

Correction Détaillée : 1. Posons les variables intermédiaires pour décomposer le graphe de calcul : $z_1 = w_5x + b_5$ $a_1 = \sigma(z_1)$ $z_2 = w_6a_1 + b_6$ $f = \sigma(z_2)$

2. Nous cherchons à évaluer $\frac{\partial f}{\partial w_5}$. Selon la règle de composition (Chain Rule) :

$$\frac{\partial f}{\partial w_5} = \frac{\partial f}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_5}$$

3. Évaluons chaque terme séparément : - $\frac{\partial f}{\partial z_2} = \sigma'(z_2)$ - $\frac{\partial z_2}{\partial a_1} = w_6$ - $\frac{\partial a_1}{\partial z_1} = \sigma'(z_1)$ - $\frac{\partial z_1}{\partial w_5} = x$

4. Par multiplication, le résultat final rigoureux sans aucune ellipse est :

$$\frac{\partial f}{\partial w_5} = \sigma'(z_2) \cdot w_6 \cdot \sigma'(z_1) \cdot x$$

Matrices Jacobiennes et Backpropagation - Cas 6 ★★★★★

Énoncé : Considérons une variante du perceptron simple défini par la fonction scalaire :

$$f(x) = \sigma(w_7 \cdot \sigma(w_6x + b_6) + b_7)$$

avec $x, w_6, w_7, b_6, b_7 \in \mathbb{R}$. Calculer analytiquement la dérivée partielle de f par rapport à w_6 , notée $\frac{\partial f}{\partial w_6}$, en appliquant de manière rigoureuse le théorème de dérivation des fonctions composées.

Correction Détaillée : 1. Posons les variables intermédiaires pour décomposer le graphe de calcul : $z_1 = w_6x + b_6$ $a_1 = \sigma(z_1)$ $z_2 = w_7a_1 + b_7$ $f = \sigma(z_2)$

2. Nous cherchons à évaluer $\frac{\partial f}{\partial w_6}$. Selon la règle de composition (Chain Rule) :

$$\frac{\partial f}{\partial w_6} = \frac{\partial f}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_6}$$

3. Évaluons chaque terme séparément : - $\frac{\partial f}{\partial z_2} = \sigma'(z_2)$ - $\frac{\partial z_2}{\partial a_1} = w_7$ - $\frac{\partial a_1}{\partial z_1} = \sigma'(z_1)$ - $\frac{\partial z_1}{\partial w_6} = x$

4. Par multiplication, le résultat final rigoureux sans aucune ellipse est :

$$\frac{\partial f}{\partial w_6} = \sigma'(z_2) \cdot w_7 \cdot \sigma'(z_1) \cdot x$$

Matrices Jacobiennes et Backpropagation - Cas 7 ★★☆☆

Énoncé : Considérons une variante du perceptron simple défini par la fonction scalaire :

$$f(x) = \sigma(w_8 \cdot \sigma(w_7x + b_7) + b_8)$$

avec $x, w_7, w_8, b_7, b_8 \in \mathbb{R}$. Calculer analytiquement la dérivée partielle de f par rapport à w_7 , notée $\frac{\partial f}{\partial w_7}$, en appliquant de manière rigoureuse le théorème de dérivation des fonctions composées.

Correction Détaillée : 1. Posons les variables intermédiaires pour décomposer le graphe de calcul : $z_1 = w_7x + b_7$ $a_1 = \sigma(z_1)$ $z_2 = w_8a_1 + b_8$ $f = \sigma(z_2)$

2. Nous cherchons à évaluer $\frac{\partial f}{\partial w_7}$. Selon la règle de composition (Chain Rule) :

$$\frac{\partial f}{\partial w_7} = \frac{\partial f}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_7}$$

3. Évaluons chaque terme séparément : - $\frac{\partial f}{\partial z_2} = \sigma'(z_2)$ - $\frac{\partial z_2}{\partial a_1} = w_8$ - $\frac{\partial a_1}{\partial z_1} = \sigma'(z_1)$ - $\frac{\partial z_1}{\partial w_7} = x$

4. Par multiplication, le résultat final rigoureux sans aucune ellipse est :

$$\frac{\partial f}{\partial w_7} = \sigma'(z_2) \cdot w_8 \cdot \sigma'(z_1) \cdot x$$

Matrices Jacobiennes et Backpropagation - Cas 8 ★★☆☆

Énoncé : Considérons une variante du perceptron simple défini par la fonction scalaire :

$$f(x) = \sigma(w_9 \cdot \sigma(w_8x + b_8) + b_9)$$

avec $x, w_8, w_9, b_8, b_9 \in \mathbb{R}$. Calculer analytiquement la dérivée partielle de f par rapport à w_8 , notée $\frac{\partial f}{\partial w_8}$, en appliquant de manière rigoureuse le théorème de dérivation des fonctions composées.

Correction Détaillée : 1. Posons les variables intermédiaires pour décomposer le graphe de calcul : $z_1 = w_8x + b_8$ $a_1 = \sigma(z_1)$ $z_2 = w_9a_1 + b_9$ $f = \sigma(z_2)$

2. Nous cherchons à évaluer $\frac{\partial f}{\partial w_8}$. Selon la règle de composition (Chain Rule) :

$$\frac{\partial f}{\partial w_8} = \frac{\partial f}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_8}$$

3. Évaluons chaque terme séparément : - $\frac{\partial f}{\partial z_2} = \sigma'(z_2)$ - $\frac{\partial z_2}{\partial a_1} = w_9$ - $\frac{\partial a_1}{\partial z_1} = \sigma'(z_1)$ - $\frac{\partial z_1}{\partial w_8} = x$

4. Par multiplication, le résultat final rigoureux sans aucune ellipse est :

$$\frac{\partial f}{\partial w_8} = \sigma'(z_2) \cdot w_9 \cdot \sigma'(z_1) \cdot x$$

Matrices Jacobiennes et Backpropagation - Cas 9 ★★★★★

Énoncé : Considérons une variante du perceptron simple défini par la fonction scalaire :

$$f(x) = \sigma(w_{10} \cdot \sigma(w_9 x + b_9) + b_{10})$$

avec $x, w_9, w_{10}, b_9, b_{10} \in \mathbb{R}$. Calculer analytiquement la dérivée partielle de f par rapport à w_9 , notée $\frac{\partial f}{\partial w_9}$, en appliquant de manière rigoureuse le théorème de dérivation des fonctions composées.

Correction Détaillée : 1. Posons les variables intermédiaires pour décomposer le graphe de calcul : $z_1 = w_9 x + b_9$ $a_1 = \sigma(z_1)$ $z_2 = w_{10} a_1 + b_{10}$ $f = \sigma(z_2)$

2. Nous cherchons à évaluer $\frac{\partial f}{\partial w_9}$. Selon la règle de composition (Chain Rule) :

$$\frac{\partial f}{\partial w_9} = \frac{\partial f}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_9}$$

3. Évaluons chaque terme séparément : - $\frac{\partial f}{\partial z_2} = \sigma'(z_2)$ - $\frac{\partial z_2}{\partial a_1} = w_{10}$ - $\frac{\partial a_1}{\partial z_1} = \sigma'(z_1)$ - $\frac{\partial z_1}{\partial w_9} = x$

4. Par multiplication, le résultat final rigoureux sans aucune ellipse est :

$$\frac{\partial f}{\partial w_9} = \sigma'(z_2) \cdot w_{10} \cdot \sigma'(z_1) \cdot x$$

Matrices Jacobiennes et Backpropagation - Cas 10 ★★☆☆☆

Énoncé : Considérons une variante du perceptron simple défini par la fonction scalaire :

$$f(x) = \sigma(w_{11} \cdot \sigma(w_{10} x + b_{10}) + b_{11})$$

avec $x, w_{10}, w_{11}, b_{10}, b_{11} \in \mathbb{R}$. Calculer analytiquement la dérivée partielle de f par rapport à w_{10} , notée $\frac{\partial f}{\partial w_{10}}$, en appliquant de manière rigoureuse le théorème de dérivation des fonctions composées.

Correction Détaillée : 1. Posons les variables intermédiaires pour décomposer le graphe de calcul : $z_1 = w_{10} x + b_{10}$ $a_1 = \sigma(z_1)$ $z_2 = w_{11} a_1 + b_{11}$ $f = \sigma(z_2)$

2. Nous cherchons à évaluer $\frac{\partial f}{\partial w_{10}}$. Selon la règle de composition (Chain Rule) :

$$\frac{\partial f}{\partial w_{10}} = \frac{\partial f}{\partial z_2} \cdot \frac{\partial z_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{10}}$$

3. Évaluons chaque terme séparément : - $\frac{\partial f}{\partial z_2} = \sigma'(z_2)$ - $\frac{\partial z_2}{\partial a_1} = w_{11}$ - $\frac{\partial a_1}{\partial z_1} = \sigma'(z_1)$ - $\frac{\partial z_1}{\partial w_{10}} = x$

4. Par multiplication, le résultat final rigoureux sans aucune ellipse est :

$$\frac{\partial f}{\partial w_{10}} = \sigma'(z_2) \cdot w_{11} \cdot \sigma'(z_1) \cdot x$$

Troisième partie

Travaux Pratiques et Simulations Algorithmiques

Rétropropagation Numpy - Architecture 1 ★★☆☆

Objectif : Implémenter la propagation avant et la rétropropagation des erreurs pour un réseau de neurones multicouche en Python pur. Dans ce TP, l'architecture testée a 3 couches cachées.

```
1 import numpy as np
2
3 def sigmoid(x):
4     return 1.0 / (1.0 + np.exp(-x))
5
6 def sigmoid_prime(x):
7     s = sigmoid(x)
8     return s * (1.0 - s)
9
10 class NeuralNetwork_1:
11     def __init__(self, layers):
12         self.num_layers = len(layers)
13         self.biases = [np.random.randn(y, 1) for y in layers[1:]]
14         self.weights = [np.random.randn(y, x) for x, y in zip(layers[:-1], layers[1:])]
15
16     def backprop(self, x, y):
17         nabla_b = [np.zeros(b.shape) for b in self.biases]
18         nabla_w = [np.zeros(w.shape) for w in self.weights]
19
20         # Propagation avant
21         activation = x
22         activations = [x]
23         zs = []
24         for b, w in zip(self.biases, self.weights):
25             z = np.dot(w, activation) + b
26             zs.append(z)
27             activation = sigmoid(z)
28             activations.append(activation)
29
30         # Rétropropagation - Couche de sortie
31         delta = (activations[-1] - y) * sigmoid_prime(zs[-1])
32         nabla_b[-1] = delta
33         nabla_w[-1] = np.dot(delta, activations[-2].transpose())
34
35         # Rétropropagation - Couches cachées
36         for l in range(2, self.num_layers):
37             z = zs[-l]
38             sp = sigmoid_prime(z)
39             delta = np.dot(self.weights[-l+1].transpose(), delta) * sp
40             nabla_b[-l] = delta
41             nabla_w[-l] = np.dot(delta, activations[-l-1].transpose())
42
43         return nabla_b, nabla_w
```

```

44
45 # TP 1 : Test d'intégrité avec l'architecture spécifique au TP 1
46 # TP 1 : On utilise 3 couches cachées pour vérifier la propagation sur
    une plus grande profondeur
47 layers_config = [2] + [3] * 3 + [1]
48 nn = NeuralNetwork_1(layers_config)
49 x = np.array([[0.5], [0.1]])
50 y = np.array([[1.0]])
51 nabla_b, nabla_w = nn.backprop(x, y)
52 assert len(nabla_w) == len(layers_config) - 1

```

Rétropropagation Numpy - Architecture 2 ★★★☆☆

Objectif : Implémenter la propagation avant et la rétropropagation des erreurs pour un réseau de neurones multicouche en Python pur. Dans ce TP, l'architecture testée a 4 couches cachées.

```

1 import numpy as np
2
3 def sigmoid(x):
4     return 1.0 / (1.0 + np.exp(-x))
5
6 def sigmoid_prime(x):
7     s = sigmoid(x)
8     return s * (1.0 - s)
9
10 class NeuralNetwork_2:
11     def __init__(self, layers):
12         self.num_layers = len(layers)
13         self.biases = [np.random.randn(y, 1) for y in layers[1:]]
14         self.weights = [np.random.randn(y, x) for x, y in zip(layers
15                                    [:-1], layers[1:])]
16
17     def backprop(self, x, y):
18         nabla_b = [np.zeros(b.shape) for b in self.biases]
19         nabla_w = [np.zeros(w.shape) for w in self.weights]
20
21         # Propagation avant
22         activation = x
23         activations = [x]
24         zs = []
25         for b, w in zip(self.biases, self.weights):
26             z = np.dot(w, activation) + b
27             zs.append(z)
28             activation = sigmoid(z)
29             activations.append(activation)
30
31         # Rétropropagation - Couche de sortie
32         delta = (activations[-1] - y) * sigmoid_prime(zs[-1])
33         nabla_b[-1] = delta
34         nabla_w[-1] = np.dot(delta, activations[-2].transpose())
35
36         # Rétropropagation - Couches cachées

```

```

36         for l in range(2, self.num_layers):
37             z = zs[-1]
38             sp = sigmoid_prime(z)
39             delta = np.dot(self.weights[-1+1].transpose(), delta) * sp
40             nabla_b[-1] = delta
41             nabla_w[-1] = np.dot(delta, activations[-1-1].transpose())
42
43         return nabla_b, nabla_w
44
45     # TP 2 : Test d'intégrité avec l'architecture spécifique au TP 2
46     # TP 2 : On utilise 4 couches cachées pour vérifier la propagation sur
47             une plus grande profondeur
48     layers_config = [2] + [3] * 4 + [1]
49     nn = NeuralNetwork_2(layers_config)
50     x = np.array([[0.5], [0.1]])
51     y = np.array([[1.0]])
52     nabla_b, nabla_w = nn.backprop(x, y)
53     assert len(nabla_w) == len(layers_config) - 1

```

Rétropropagation Numpy - Architecture 3 ★★★★★

Objectif : Implémenter la propagation avant et la rétropropagation des erreurs pour un réseau de neurones multicouche en Python pur. Dans ce TP, l'architecture testée a 5 couches cachées.

```

1  import numpy as np
2
3  def sigmoid(x):
4      return 1.0 / (1.0 + np.exp(-x))
5
6  def sigmoid_prime(x):
7      s = sigmoid(x)
8      return s * (1.0 - s)
9
10 class NeuralNetwork_3:
11     def __init__(self, layers):
12         self.num_layers = len(layers)
13         self.biases = [np.random.randn(y, 1) for y in layers[1:]]
14         self.weights = [np.random.randn(y, x) for x, y in zip(layers[:-1], layers[1:])]
15
16     def backprop(self, x, y):
17         nabla_b = [np.zeros(b.shape) for b in self.biases]
18         nabla_w = [np.zeros(w.shape) for w in self.weights]
19
20         # Propagation avant
21         activation = x
22         activations = [x]
23         zs = []
24         for b, w in zip(self.biases, self.weights):
25             z = np.dot(w, activation) + b
26             zs.append(z)
27             activation = sigmoid(z)

```

```

28         activations.append(activation)
29
30     # Rétropropagation - Couche de sortie
31     delta = (activations[-1] - y) * sigmoid_prime(zs[-1])
32     nabla_b[-1] = delta
33     nabla_w[-1] = np.dot(delta, activations[-2].transpose())
34
35     # Rétropropagation - Couches cachées
36     for l in range(2, self.num_layers):
37         z = zs[-l]
38         sp = sigmoid_prime(z)
39         delta = np.dot(self.weights[-l+1].transpose(), delta) * sp
40         nabla_b[-l] = delta
41         nabla_w[-l] = np.dot(delta, activations[-l-1].transpose())
42
43     return nabla_b, nabla_w
44
45 # TP 3 : Test d'intégrité avec l'architecture spécifique au TP 3
46 # TP 3 : On utilise 5 couches cachées pour vérifier la propagation sur
une plus grande profondeur
47 layers_config = [2] + [3] * 5 + [1]
48 nn = NeuralNetwork_3(layers_config)
49 x = np.array([[0.5], [0.1]])
50 y = np.array([[1.0]])
51 nabla_b, nabla_w = nn.backprop(x, y)
52 assert len(nabla_w) == len(layers_config) - 1

```

Rétropropagation Numpy - Architecture 4 ★★☆☆

Objectif : Implémenter la propagation avant et la rétropropagation des erreurs pour un réseau de neurones multicouche en Python pur. Dans ce TP, l'architecture testée a 6 couches cachées.

```

1 import numpy as np
2
3 def sigmoid(x):
4     return 1.0 / (1.0 + np.exp(-x))
5
6 def sigmoid_prime(x):
7     s = sigmoid(x)
8     return s * (1.0 - s)
9
10 class NeuralNetwork_4:
11     def __init__(self, layers):
12         self.num_layers = len(layers)
13         self.biases = [np.random.randn(y, 1) for y in layers[1:]]
14         self.weights = [np.random.randn(y, x) for x, y in zip(layers[:-1], layers[1:])]
15
16     def backprop(self, x, y):
17         nabla_b = [np.zeros(b.shape) for b in self.biases]
18         nabla_w = [np.zeros(w.shape) for w in self.weights]
19

```

```

20     # Propagation avant
21     activation = x
22     activations = [x]
23     zs = []
24     for b, w in zip(self.biases, self.weights):
25         z = np.dot(w, activation) + b
26         zs.append(z)
27         activation = sigmoid(z)
28         activations.append(activation)
29
30     # Rétropropagation - Couche de sortie
31     delta = (activations[-1] - y) * sigmoid_prime(zs[-1])
32     nabla_b[-1] = delta
33     nabla_w[-1] = np.dot(delta, activations[-2].transpose())
34
35     # Rétropropagation - Couches cachées
36     for l in range(2, self.num_layers):
37         z = zs[-l]
38         sp = sigmoid_prime(z)
39         delta = np.dot(self.weights[-l+1].transpose(), delta) * sp
40         nabla_b[-l] = delta
41         nabla_w[-l] = np.dot(delta, activations[-l-1].transpose())
42
43     return nabla_b, nabla_w
44
45 # TP 4 : Test d'intégrité avec l'architecture spécifique au TP 4
46 # TP 4 : On utilise 6 couches cachées pour vérifier la propagation sur
une plus grande profondeur
47 layers_config = [2] + [3] * 6 + [1]
48 nn = NeuralNetwork_4(layers_config)
49 x = np.array([[0.5], [0.1]])
50 y = np.array([[1.0]])
51 nabla_b, nabla_w = nn.backprop(x, y)
52 assert len(nabla_w) == len(layers_config) - 1

```

Rétropropagation Numpy - Architecture 5 ★★★★★

Objectif : Implémenter la propagation avant et la rétropropagation des erreurs pour un réseau de neurones multicouche en Python pur. Dans ce TP, l'architecture testée a 7 couches cachées.

```

1  import numpy as np
2
3  def sigmoid(x):
4      return 1.0 / (1.0 + np.exp(-x))
5
6  def sigmoid_prime(x):
7      s = sigmoid(x)
8      return s * (1.0 - s)
9
10 class NeuralNetwork_5:
11     def __init__(self, layers):
12         self.num_layers = len(layers)

```

```

13     self.biases = [np.random.randn(y, 1) for y in layers[1:]]
14     self.weights = [np.random.randn(y, x) for x, y in zip(layers
15         [-1], layers[1:])]
16
17     def backprop(self, x, y):
18         nabla_b = [np.zeros(b.shape) for b in self.biases]
19         nabla_w = [np.zeros(w.shape) for w in self.weights]
20
21         # Propagation avant
22         activation = x
23         activations = [x]
24         zs = []
25         for b, w in zip(self.biases, self.weights):
26             z = np.dot(w, activation) + b
27             zs.append(z)
28             activation = sigmoid(z)
29             activations.append(activation)
30
31         # Rétropropagation - Couche de sortie
32         delta = (activations[-1] - y) * sigmoid_prime(zs[-1])
33         nabla_b[-1] = delta
34         nabla_w[-1] = np.dot(delta, activations[-2].transpose())
35
36         # Rétropropagation - Couches cachées
37         for l in range(2, self.num_layers):
38             z = zs[-l]
39             sp = sigmoid_prime(z)
40             delta = np.dot(self.weights[-l+1].transpose(), delta) * sp
41             nabla_b[-l] = delta
42             nabla_w[-l] = np.dot(delta, activations[-l-1].transpose())
43
44         return nabla_b, nabla_w
45
46     # TP 5 : Test d'intégrité avec l'architecture spécifique au TP 5
47     # TP 5 : On utilise 7 couches cachées pour vérifier la propagation sur
48     une plus grande profondeur
49     layers_config = [2] + [3] * 7 + [1]
50     nn = NeuralNetwork_5(layers_config)
51     x = np.array([[0.5], [0.1]])
52     y = np.array([[1.0]])
53     nabla_b, nabla_w = nn.backprop(x, y)
54     assert len(nabla_w) == len(layers_config) - 1

```